

ENGINEERING REBOOT 2026

Day 33 Engineering Field Manual

Week 07 – Software Architecture & Object-Oriented Programming

Version: 1.0 (Locked)

Status: Approved for Implementation

Curriculum Continuity

Planning Authority

- Week 07 Master Workbook (Locked)

Previous Day (Completed)

- Classes and Objects
- Constructors
- Instance Methods
- Encapsulation

Curriculum Progress

Completed

- ✓ Classes
- ✓ Objects
- ✓ Constructors
- ✓ Instance Methods
- ✓ Encapsulation

Today's Topic

→ Composition

Deferred

- ☐ OOP Refactoring
- ☐ Week Integration Project
- ☐ Inheritance (*Future Week*)
- ☐ Polymorphism (*Future Week*)

Scope Statement

This Field Manual introduces **Composition** only. No inheritance, polymorphism, or advanced OOP concepts are introduced in today's implementation.

Mission

Today's objective is to learn how multiple objects collaborate to solve a larger problem.

Rather than creating increasingly complex classes, software systems are often built from **small, focused classes working together**. This design principle is known as **composition**, and it models "**has-a**" relationships between objects.

Learning Objectives

By the end of today's session, you should be able to:

Technical

- Explain composition.
- Model "has-a" relationships.
- Create classes that collaborate.
- Store objects inside another object.
- Delegate responsibilities appropriately.
- Manage collections of objects.

Engineering

Understand:

- Why composition improves maintainability.
 - Why responsibilities should remain localized.
 - Why large classes should be decomposed into smaller collaborating objects.
-

Core Concepts

Today's topics:

- Composition
 - Object Collaboration
 - Has-a Relationships
 - Delegation
 - Aggregation of Objects
 - Managing Object Collections
-

Exercise 1 — Building a Simple Class

File

`book.py`

Objective

Create a `Book` class containing:

- `title`
- `author`
- `ISBN`

Implement:

- `constructor`
 - `display()`
-

Exercise 2 — Composition

File

`library.py`

Create a `Library` class that maintains a collection of `Book` objects.

Implement:

- `add_book()`
- `display_books()`

The library should store **Book objects**, not dictionaries.

Exercise 3 — Searching Objects

Extend `Library`.

Implement:

- `find_book_by_title()`
- `find_book_by_isbn()`

Return the matching `Book` object if found.

Exercise 4 — Removing Objects

Extend `Library`.

Implement:

`remove_book()`

Requirements:

- Remove using ISBN.
 - Return success/failure.
 - Display an appropriate status message.
-

Mini Project

Library Management System

Files

`book.py`

```
library.py  
main.py
```

System Overview

Demonstrate composition by implementing:

```
Library  
  has many  
    ↓  
  Book
```

The `Library` object owns and manages a collection of `Book` objects.

Book Class

Attributes

- title
- author
- ISBN

Methods

- `display()`
-

Library Class

Maintain:

```
self._books
```

Methods

```
add_book()
```

```
remove_book()
```

```
find_book()
```

```
display_books()
```

```
total_books()
```

Main Program

Create several `Book` objects.

Instantiate one `Library`.

Demonstrate:

- adding books
- listing books
- searching
- removing books
- displaying the updated collection
- total number of books

Validation Requirements

Your implementation should correctly handle:

- ✓ Adding multiple books
- ✓ Searching for an existing book
- ✓ Searching for a missing book
- ✓ Removing an existing book
- ✓ Attempting to remove a nonexistent book
- ✓ Accurate book count
- ✓ No runtime exceptions during normal operation

Engineering Discussion

Observe how responsibilities are separated.

Book

Responsible only for representing a book.

Library

Responsible for managing books.

Main Program

Responsible for coordinating user actions.

This separation is the essence of composition.

Notes.md Guidance

Document:

- Definition of composition.
 - Difference between composition and encapsulation.
 - Difference between "has-a" and "is-a" relationships.
 - Why composition improves software design.
 - Lessons learned while implementing collaborating classes.
-

Journal.md Guidance

Reflect on:

- How composition differs from the single-class design of Day 32.
 - Challenges encountered while coordinating multiple classes.
 - Whether the design felt more modular than previous exercises.
 - Real-world examples where composition is preferable.
-

Evidence Checklist

Capture:

- Exercise 1 source
 - Exercise 2 source
 - Exercise 3 source
 - Exercise 4 source
 - Mini-project source
 - Successful execution
 - Search demonstration
 - Removal demonstration
 - Git operations
-

Completion Criteria

Day 33 is complete when:

- All exercises are implemented.
- The mini-project demonstrates composition through collaborating classes.
- Validation scenarios execute successfully.
- Notes.md completed.
- Journal.md completed.
- Git changes committed and pushed.
- Evidence archived.